
Universidade de Passo Fundo
Análise e Desenvolvimento de Sistemas
Programação e Otimização de Banco de Dados (Teórico 1)

NOME: Jefferson H. Santos
MATRÍCULA: 210017

Atividade EaD 3: SP Sistema bancário

VISÃO GERAL

EAD3_Correntista = #Código, nome

EAD3_Contas = #Número, saldo, @Código do correntista

EAD3_Movimentacao=#Número, data, valor, natureza (débito/crédito), @numero da conta

OBJETIVOS

1. * Insere um novo cliente e abre uma conta associada ao mesmo
 - a) abreConta(nome)
2. * Deposita valor em uma conta
 - b) deposita(Numero conta, valor)
3. * Saca valor de uma conta
 - c) saca(Numero conta, valor)
4. * Transfere valor de uma conta de origem para uma conta de destino
 - d) transfere(Conta origem, conta destino, valor)

OBSERVAÇÕES:

- a cada operação realizada (saque, depósito, etc) um registro de movimentação deve ser inserido automaticamente pelas funções na tabela de Movimentação, descrevendo a operação se houve débito ou crédito
- não poderá haver saque de valor superior ao disponível no saldo da conta

```
select * from ead3_contas

select * from ead3_correntista

select * from ead3_movimentacao

SET SERVEROUTPUT ON;

-- inserção de tabelas:

-- 1. Criação da tabela EAD3_CORRENTISTA

CREATE TABLE EAD3_CORRENTISTA (

    codigo NUMBER GENERATED BY DEFAULT AS IDENTITY,

    nome VARCHAR2(100) NOT NULL,

    --

    CONSTRAINT pk_ead3_correntista PRIMARY KEY (codigo)

);

-- 2. Criação da tabela EAD3_CONTAS

CREATE TABLE EAD3_CONTAS (

    numero NUMBER,
```

```
saldo NUMBER(15, 2) DEFAULT 0.00 NOT NULL,

codigo_correntista NUMBER NOT NULL,

--

CONSTRAINT pk_ead3_contas PRIMARY KEY (numero),

CONSTRAINT fk_ead3_contas_correntista FOREIGN KEY (codigo_correntista)

REFERENCES EAD3_CORRENTISTA(codigo) ON DELETE CASCADE

);

-- 3. Criação da tabela EAD3_MOVIMENTACAO

CREATE TABLE EAD3_MOVIMENTACAO (

    id_movimentacao NUMBER GENERATED BY DEFAULT AS IDENTITY,

    numero_mov NUMBER NOT NULL,

    data DATE DEFAULT SYSDATE NOT NULL,

    valor NUMBER(15, 2) NOT NULL,

    natureza CHAR(1) NOT NULL,

    numero_conta NUMBER NOT NULL,

    --

    CONSTRAINT pk_ead3_movimentacao PRIMARY KEY (id_movimentacao),

    CONSTRAINT fk_ead3_mov_contas FOREIGN KEY (numero_conta)

REFERENCES EAD3_CONTAS(numero) ON DELETE CASCADE,

    CONSTRAINT chk_ead3_natureza CHECK (natureza IN ('D', 'C'))

);
```

```
--* Insere um novo cliente e abre uma conta associada ao mesmo
--a) abreConta(nome)

CREATE OR REPLACE PROCEDURE abreConta (
    p_nome IN VARCHAR2
) AS
    v_novo_codigo NUMBER;
    v_novo_numero_conta NUMBER;
BEGIN
    -- Passo 1: Inserir o correntista e capturar o ID gerado
    automaticamente

    INSERT INTO EAD3_CORRENTISTA (nome)
    VALUES (p_nome)
    RETURNING codigo INTO v_novo_codigo;

    -- Passo 2: Calcular o próximo número de conta disponível

    SELECT NVL(MAX(numero), 1000) + 1
    INTO v_novo_numero_conta
    FROM EAD3_CONTAS;

    -- Passo 3: Criar a conta associada ao novo correntista (saldo inicial
    0)
```

```
INSERT INTO EAD3_CONTAS (numero, saldo, codigo_correntista)

VALUES (v_novo_numero_conta, 0.00, v_novo_codigo);


-- Importante: Confirmar as duas inserções no banco

COMMIT;


-- Mensagem de feedback no console do SQL Developer

DBMS_OUTPUT.PUT_LINE('Cliente ' || p_nome || ' cadastrado. Conta
numero: ' || v_novo_numero_conta);

EXCEPTION

    WHEN OTHERS THEN

        ROLLBACK; -- Desfaz tudo se der qualquer erro (garante que não
crie cliente sem conta)

        RAISE_APPLICATION_ERROR(-20001, 'Erro ao abrir a conta: ' ||
SQLERRM);

END abreConta;

/


-- Executa a procedure passando o nome do cliente

EXEC abreConta('Angelo Preto');


--* Deposita valor em uma conta
```

```
--b) deposita(Numero conta, valor)

CREATE OR REPLACE PROCEDURE deposita (

    p_numero_conta IN NUMBER,

    p_valor          IN NUMBER

) AS

    v_proximo_num_mov NUMBER;

    v_existe_conta     NUMBER;

BEGIN

    -- Passo 1: Validar se a conta realmente existe no banco

    SELECT COUNT(*)

    INTO v_existe_conta

    FROM EAD3_CONTAS

    WHERE numero = p_numero_conta;

    IF v_existe_conta = 0 THEN

        -- Dispara um erro customizado caso a conta não exista

        RAISE_APPLICATION_ERROR(-20002, 'Erro: A conta ' || p_numero_conta
|| ' nao existe.');
```

```
    END IF;

    -- Passo 2: Validar se o valor do depósito é maior que zero

    IF p_valor <= 0 THEN

        RAISE_APPLICATION_ERROR(-20003, 'Erro: O valor do deposito deve
ser maior que zero.');
```

```
END IF;

-- Passo 3: Atualizar o saldo da conta somando o valor recebido

UPDATE EAD3_CONTAS

SET saldo = saldo + p_valor

WHERE numero = p_numero_conta;

-- Passo 4: Descobrir o próximo número de movimentação para ESTA conta

SELECT NVL(MAX(numero_mov), 0) + 1

INTO v_proximo_num_mov

FROM EAD3_MOVIMENTACAO

WHERE numero_conta = p_numero_conta;

-- Passo 5: Inserir o registro de Crédito ('C') na tabela de
movimentações

INSERT INTO EAD3_MOVIMENTACAO (numero_mov, data, valor, natureza,
numero_conta)

VALUES (v_proximo_num_mov, SYSDATE, p_valor, 'C', p_numero_conta);

-- Confirmar o depósito e a movimentação no banco

COMMIT;

DBMS_OUTPUT.PUT_LINE('Deposito de R$ ' || p_valor || ' realizado com
sucesso na conta ' || p_numero_conta);

EXCEPTION
```

```
        WHEN OTHERS THEN

            ROLLBACK; -- Se der qualquer erro bizarro, desfaz o update e o
insert

            RAISE_APPLICATION_ERROR(-20004, 'Erro inesperado no deposito: ' ||
SQLERRM) ;

END deposita;

/

-- Executa o depósito

EXEC deposita(1001, 500.00);

-- Verifique se o saldo mudou

SELECT * FROM EAD3_CONTAS WHERE numero = 1001;

-- Verifique se o histórico foi gravado

SELECT * FROM EAD3_MOVIMENTACAO WHERE numero_conta = 1001;

--* Saca valor de uma conta

--c) saca(Numero conta, valor)
```



```
CREATE OR REPLACE PROCEDURE saca (  
    p_numero_conta IN NUMBER,  
    p_valor        IN NUMBER  
) AS  
    v_saldo_atual   NUMBER(15, 2);  
    v_proximo_num_mov NUMBER;  
    v_existe_conta  NUMBER;  
BEGIN  
    -- Passo 1: Validar se a conta existe  
    SELECT COUNT(*)  
    INTO v_existe_conta  
    FROM EAD3_CONTAS  
    WHERE numero = p_numero_conta;  
  
    IF v_existe_conta = 0 THEN  
        RAISE_APPLICATION_ERROR(-20002, 'Erro: A conta ' || p_numero_conta  
|| ' nao existe.');    END IF;  
  
    -- Passo 2: Validar se o valor do saque é positivo  
    IF p_valor <= 0 THEN  
        RAISE_APPLICATION_ERROR(-20003, 'Erro: O valor do saque deve ser  
maior que zero.');    END IF;
```

```
-- Passo 3: Buscar o saldo atual da conta para validação

SELECT saldo

INTO v_saldo_atual

FROM EAD3_CONTAS

WHERE numero = p_numero_conta;

-- Passo 4: REGRA DE NEGÓCIO - Bloquear se o saldo for insuficiente

IF v_saldo_atual < p_valor THEN

    RAISE_APPLICATION_ERROR(-20005, 'Erro: Saldo insuficiente. Saldo
atual: R$ ' || v_saldo_atual);

END IF;

-- Passo 5: Atualizar o saldo subtraindo o valor sacado

UPDATE EAD3_CONTAS

SET saldo = saldo - p_valor

WHERE numero = p_numero_conta;

-- Passo 6: Descobrir o próximo número de movimentação para a conta

SELECT NVL(MAX(numero_mov), 0) + 1

INTO v_proximo_num_mov

FROM EAD3_MOVIMENTACAO

WHERE numero_conta = p_numero_conta;

-- Passo 7: Inserir a movimentação como Débito ('D')
```

```
INSERT INTO EAD3_MOVIMENTACAO (numero_mov, data, valor, natureza,
numero_conta)

VALUES (v_proximo_num_mov, SYSDATE, p_valor, 'D', p_numero_conta);

-- Confirmar a transação

COMMIT;

DBMS_OUTPUT.PUT_LINE('Saque de R$ ' || p_valor || ' realizado com
sucesso na conta ' || p_numero_conta);

EXCEPTION

    WHEN OTHERS THEN

        ROLLBACK; -- Se falhar, garante que o dinheiro não "suma" da conta

        RAISE_APPLICATION_ERROR(-20004, 'Erro inesperado no saque: ' ||
SQLERRM);

END saca;

/

-- Cenário 1: Tentando sacar mais do que tem (Deve dar erro de saldo
insuficiente)

EXEC saca(1001, 1000.00);

-- Cenário 2: Sacando um valor permitido (Deve funcionar)

EXEC saca(1001, 150.00);

-- Verificando os resultados
```

```
SELECT * FROM EAD3_CONTAS WHERE numero = 1001;

SELECT * FROM EAD3_MOVIMENTACAO WHERE numero_conta = 1001;


--* Transfere valor de uma conta de origem para uma conta de destino
--d) transfere(Conta origem, conta destino, valor)


CREATE OR REPLACE PROCEDURE transfere (

    p_conta_origem  IN NUMBER,

    p_conta_destino IN NUMBER,

    p_valor         IN NUMBER

) AS

    v_existe_origem  NUMBER;

    v_existe_destino NUMBER;

BEGIN

    -- Passo 1: Validar se a conta de origem e destino não são iguais

    IF p_conta_origem = p_conta_destino THEN

        RAISE_APPLICATION_ERROR(-20006, 'Erro: A conta de origem nao pode
ser igual a conta de destino.');
```

```
    END IF;

    -- Passo 2: Validar se a conta de origem existe

    SELECT COUNT(*) INTO v_existe_origem FROM EAD3_CONTAS WHERE numero =
p_conta_origem;
```

```
IF v_existe_origem = 0 THEN

    RAISE_APPLICATION_ERROR(-20002, 'Erro: Conta de origem (' ||
p_conta_origem || ') nao existe.');
```

END IF;

-- Passo 3: Validar se a conta de destino existe

```
SELECT COUNT(*) INTO v_existe_destino FROM EAD3_CONTAS WHERE numero =
p_conta_destino;
```

```
IF v_existe_destino = 0 THEN

    RAISE_APPLICATION_ERROR(-20002, 'Erro: Conta de destino (' ||
p_conta_destino || ') nao existe.');
```

END IF;

-- Passo 4: Executar o Saque na conta de origem

-- Se o saldo for insuficiente, a própria procedure 'saca' vai disparar o erro e parar tudo aqui.

```
saca(p_conta_origem, p_valor);
```

-- Passo 5: Executar o Depósito na conta de destino

```
deposita(p_conta_destino, p_valor);
```

-- Importante: O COMMIT aqui garante que o saque e o depósito sejam salvos JUNTOS.

-- Se o depósito falhasse por algum motivo, o ROLLBACK do bloco EXCEPTION cancelaria o saque também!

```
COMMIT;
```

```
        DBMS_OUTPUT.PUT_LINE('Transferencia de R$ ' || p_valor || ' da conta '
|| p_conta_origem || ' para a conta ' || p_conta_destino || ' realizada
com sucesso!');

EXCEPTION

    WHEN OTHERS THEN

        ROLLBACK; -- Garante que se um lado falhar, o dinheiro não suma e
nem brote do nada

        RAISE_APPLICATION_ERROR(-20007, 'Erro na transferencia: ' ||
SQLERRM);

END transfere;

/

-- Executa a transferência

EXEC transfere(1001, 1002, 100.00);

-- Verifique se os saldos foram alterados corretamente (um diminuiu, o
outro aumentou)

SELECT * FROM EAD3_CONTAS WHERE numero IN (1001, 1002);

-- Verifique se foram geradas duas movimentações automaticamente (um
Débito na 1001 e um Crédito na 1002)

SELECT * FROM EAD3_MOVIMENTACAO WHERE numero_conta IN (1001, 1002) ORDER
BY data DESC;
```

```
-----  
-----  
  
--repetindo os exec's e  
select's-----  
  
select * from ead3_contas  
  
select * from ead3_correntista  
  
select * from ead3_movimentacao  
  
  
-- Ativa a visualização de mensagens no SQL Developer  
SET SERVEROUTPUT ON;  
  
  
--a) abreConta(nome)  
  
  
-- Executa a procedure passando o nome do cliente  
EXEC abreConta('Angelo Preto');  
  
  
  
  
--b) deposita(Numero conta, valor)  
  
  
-- Executa o depósito  
EXEC deposita(1001, 500.00);  
  
-- Verifique se o saldo mudou  
SELECT * FROM EAD3_CONTAS WHERE numero = 1001;  
  
-- Verifique se o histórico foi gravado  
SELECT * FROM EAD3_MOVIMENTACAO WHERE numero_conta = 1001;
```

```
--c) saca(Numero conta, valor)

-- Cenário 1: Tentando sacar mais do que tem (Deve dar erro de saldo
insuficiente)

EXEC saca(1001, 1000.00);

-- Cenário 2: Sacando um valor permitido (Deve funcionar)

EXEC saca(1001, 150.00);

-- Verificando os resultados

SELECT * FROM EAD3_CONTAS WHERE numero = 1001;

SELECT * FROM EAD3_MOVIMENTACAO WHERE numero_conta = 1001;


--d) transfere(Conta origem, conta destino, valor)

-- Executa a transferência

EXEC transfere(1001, 1002, 100.00);

-- Verifique se os saldos foram alterados corretamente (um diminuiu, o
outro aumentou)

SELECT * FROM EAD3_CONTAS WHERE numero IN (1001, 1002);


-- Verifique se foram geradas duas movimentações automaticamente (um
Débito na 1001 e um Crédito na 1002)

SELECT * FROM EAD3_MOVIMENTACAO WHERE numero_conta IN (1001, 1002) ORDER
BY data DESC;
```